

Guia Integral del TPF

Auth, JWT, Sesiones, Roles y Pruebas

Proyecto: BackendUTN_TPF

Fecha: 2026-07-10

Documento de estudio para comprender implementacion tecnica y ejecutar pruebas completas en Postman y web.

1) Definicion funcional y objetivos

El sistema gestiona espacios colaborativos de enlaces con dos perfiles principales: owner y visitante. Un mismo usuario puede ser owner en algunos espacios y visitante en otros.

- Owner: crea espacio, administra miembros, modifica y elimina links del espacio.
- Visitante aprobado: crea links y consulta links en espacios autorizados.
- Pendiente o no autorizado: solo solicita ingreso.

Objetivo tecnico: aplicar seguridad real con JWT, bcrypt, verificacion de email y recuperacion de contrasena, manteniendo arquitectura por capas.

2) Arquitectura por capas

Se implementa separacion de responsabilidades: routes recibe HTTP, controllers gestionan req/res, services aplican reglas de negocio y permisos, repositories ejecutan SQL.

Beneficio practico: cuando una regla cambia (por ejemplo, permisos de visitante), se toca service sin romper controllers ni rutas.

Diagrama: arquitectura por capas



Guia general end-to-end (como funciona todo)

1. Inicio: Express carga dotenv, middlewares globales, conexion MySQL y rutas.
2. Registro: /api/auth/register guarda passwordHash bcrypt y emailVerificado=0.
3. Verificacion: se genera token, se guarda hash en DB y se envia correo con enlace.
4. Activacion: /api/auth/verify-email valida token y marca emailVerificado=1.
5. Login: /api/auth/login entrega accessToken JWT con expiracion.
6. Protegidos: Bearer token -> middleware JWT -> request.user -> service con reglas.
7. Persistencia: repositories ejecutan SQL sobre Usuarios, Espacios, Espacio_Usuarios y links.
8. Reset de clave: forgot/reset usa token temporal hasheado con vencimiento e invalidacion.
9. Salida UX: frontend no expone tokens; verify-email devuelve HTML amigable en navegador.

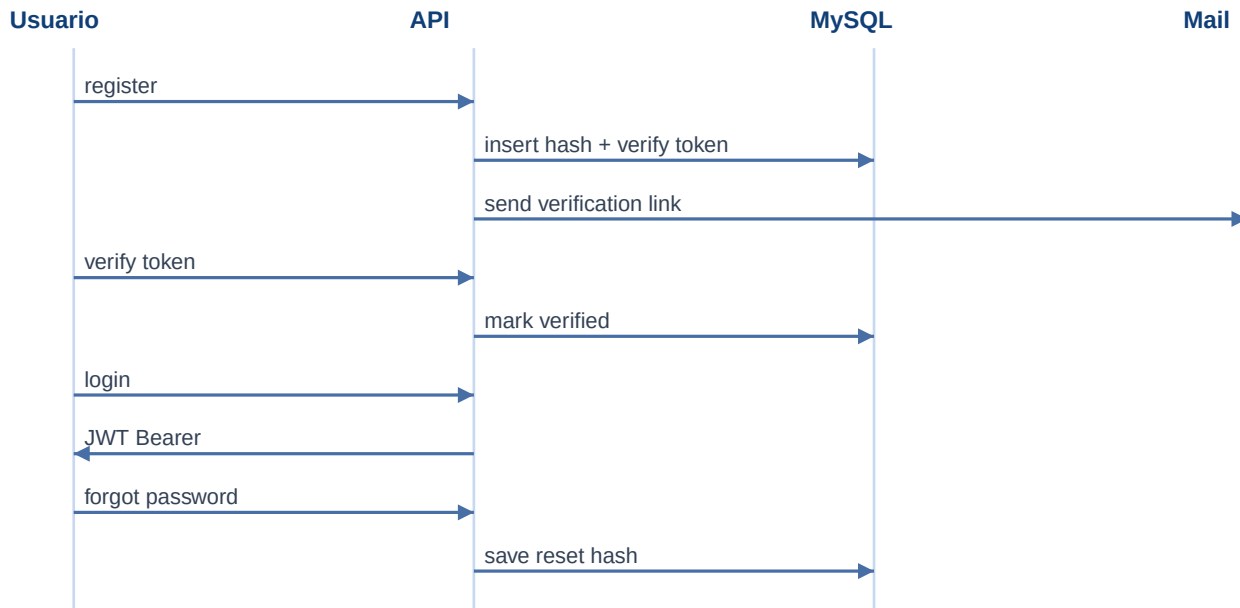
3) Autenticacion, JWT y sesion stateless

La sesion no vive en memoria del servidor. El cliente recibe un JWT y lo envia en cada request protegida como Bearer token.

- JWT incluye sub=idUsuario, email, nombre, iat y exp.
- La firma usa JWT_ACCESS_SECRET y se valida en authJwt middleware.
- Si el token expira, la API devuelve 401 y se requiere login nuevamente.
- En la app principal, cerrar sesion elimina el accessToken guardado en localStorage.

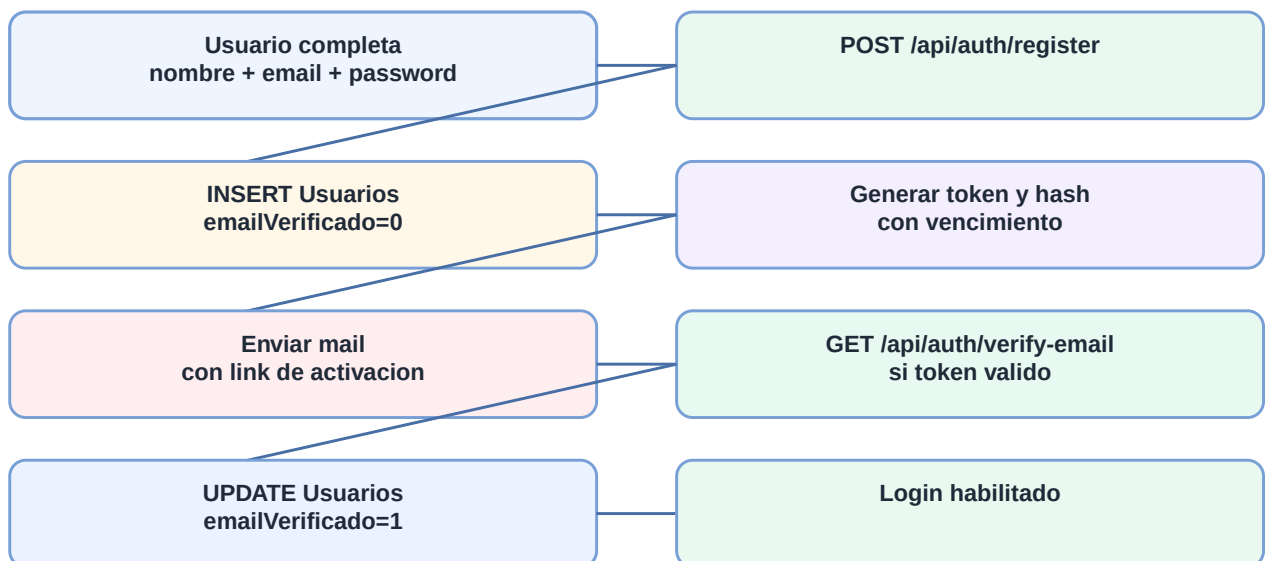
Esto simplifica escalado horizontal porque no hay estado de sesion compartido entre instancias.

Diagrama: flujo auth + JWT + reset password



Para registrar nuevos usuarios, el backend crea la cuenta con emailVerificado=0 y exige validacion desde correo antes de permitir login.

Diagrama: registracion con verificacion de email



Mensajes funcionales de login

- Usuario no registrado: "Usuario no registrado. Si es nuevo, regístrate aquí" (HTTP 401).
- Usuario registrado sin validar email: "Debe verificar su email antes de iniciar sesión" (HTTP 403).
- Objetivo: guiar al usuario con acción concreta y evitar confusión en pantalla de ingreso.

4) Seguridad aplicada y buenas practicas

- Passwords hasheadas con bcrypt.
- Tokens de verificacion/reset almacenados hasheados en DB, nunca en claro.
- Validacion de payloads con Zod para evitar entradas invalidas.
- Error handler centralizado con respuestas consistentes.
- Forgot-password responde mensaje generico para no filtrar usuarios existentes.

5) Recuperacion de contraseña (paso tecnico)

1. POST /api/auth/forgot-password recibe email.
2. Si existe el usuario, se genera token random de recuperacion.
3. Se guarda hash del token + expiracion en DB (no token plano).
4. Se envia enlace por email con el token.
5. POST /api/auth/reset-password recibe token + newPassword.
6. Se valida hash + expiracion y se actualiza passwordHash.
7. Se limpia token de reset para invalidarlo.

Resultado esperado: password vieja invalida, password nueva valida.

Justificacion: este enfoque mejora seguridad frente al envio de contraseñas por email. El sistema nunca guarda contraseñas ni tokens en claro, solo hashes y expiracion temporal.

Preguntas de comprension sobre reset por token

- Por que guardar hash del token y no token plano? Para mitigar riesgo si se expone la base de datos.
- Por que responder mensaje generico en forgot? Para evitar enumeracion de usuarios por email.
- Por que invalidar el token luego del reset? Para que el enlace sea de un solo uso.
- Por que agregar vencimiento? Para acotar ventana de uso indebido del enlace.

6) Autorizacion por rol (owner vs visitante)

Reglas en espacios

- Crear espacio: owner = usuario autenticado (request.user).
- Solicitar ingreso: idUsuario se toma del token, no del body.
- Aprobar/rechazar/expulsar: solo owner del espacio.
- Listar miembros: solo owner del espacio.

Reglas en links

- Read/List/Search: owner o visitante aprobado.
- Create: owner o visitante aprobado.
- Update/Delete: solo owner.

En create/update se envia idEspacio (numero). El backend valida permisos sobre ese espacio y persiste links con FK real a Espacios.

Mapa de endpoints (legacy -> REST)

- POST /api/crear -> POST /api/links
- PUT /api/actualizar/{id} -> PATCH /api/links/{id}
- DELETE /api/eliminar/{id} -> DELETE /api/links/{id}
- POST /api/buscar -> GET /api/links?idEspacio=...&nombre=...&comentario=...&direccion=...

Los endpoints legacy quedan solo por compatibilidad temporal; para nuevas integraciones usar /api/links.

Ciclo de solicitud de ingreso (visitante)

- Visitante solicita ingreso y queda en estado pendiente (1).
- Desde pendiente, el owner puede aprobar (2) o rechazar (3).
- Si fue aprobado, el owner puede expulsar (4).

Diagrama: ciclo de solicitud de ingreso

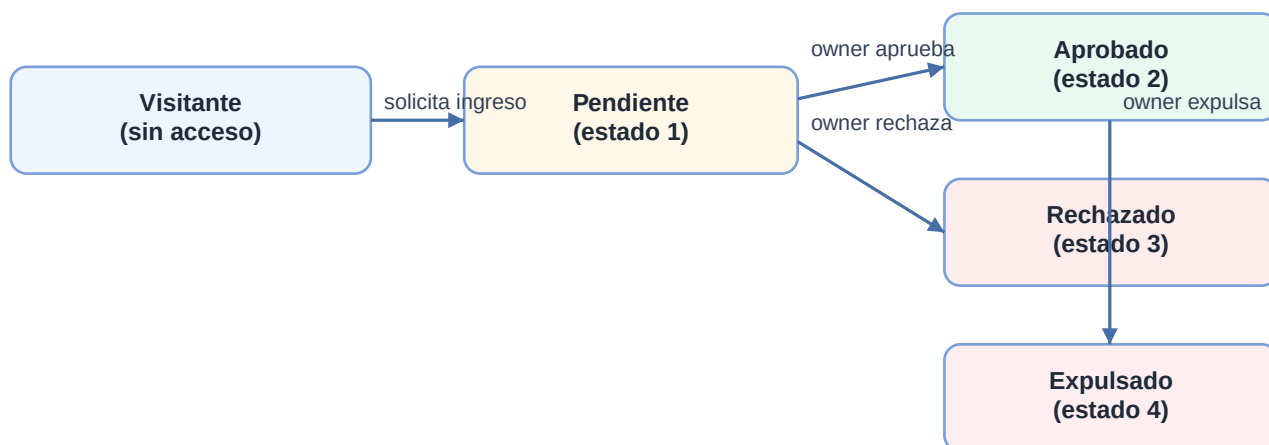


Diagrama: modelo de datos (resumen)



7) Paso a paso de pruebas en Postman (detallado)

Preparacion

1. Importar postman_collection_bookmarks.json desde el repo.
2. Variables: baseUrl, accessToken, ownerEmail, ownerPassword, ownerId, visitorEmail, visitorPassword, visitorUserId, espacioId, linkId, verifyToken, resetToken.
3. Authorization global: Bearer {{accessToken}} para endpoints protegidos.
4. Los tests de Auth guardan automaticamente verifyToken, resetToken, accessToken y user ids.

Aclaracion practica: en login/register/verify/forgot/reset usar No Auth. Bearer solo aplica a endpoints protegidos.

Excepcion de alcance para pruebas: DELETE /api/usuarios/{idUserio} se usa como cleanup y no requiere Bearer token.

Escenario A: register -> verify -> login

Request A1: POST /api/auth/register (owner).

```
{
  "nombre": "owner_demo",
  "email": "owner_demo_x@mail.com",
  "password": "12345678"
}
```

Esperado: 201 y verifyUrlDev. Extraer token de verifyUrlDev.

Request A2: GET /api/auth/verify-email?token=TOKEN_EXTRAIDO. Esperado 200.

Request A3: POST /api/auth/login. Esperado 200 y accessToken en variable global accessToken.

Repetir A1/A2/A3 para visitante y luego para owner segun el escenario de permisos.

Escenario B: owner y visitante

1. Owner crea espacio privado con POST /api/espacios.
2. Visitante solicita ingreso con POST /api/espacios/{espacioId}/solicitudes.
3. Visitante intenta crear link y debe obtener 403 si esta pendiente.
4. Owner aprueba solicitud con PUT /api/espacios/{espacioId}/solicitudes/{visitorUserId}/aprobar.
5. Visitante crea link y ahora debe obtener 201.
6. Visitante intenta modificar link y debe obtener 403.
7. Owner modifica link y debe obtener 200.

Escenario C: forgot y reset

1. POST /api/auth/forgot-password con visitorEmail.
2. El test de Postman extrae resetToken automaticamente desde resetUrlDev.
3. POST /api/auth/reset-password con token y newPassword.
4. Probar login con password vieja (401) y nueva (200).

Escenario D: limpieza de usuario para pruebas

1. Ejecutar DELETE /api/usuarios/{idUserio} para limpieza controlada de un usuario de prueba.
2. La API elimina usuario, espacios donde era owner y relaciones asociadas por cascada.
3. Tambien elimina links asociados por idEspacio para mantener integridad referencial del modelo.

Escenario D: guion con 2 cuentas Gmail de demo

Roles del ejemplo: owner = prueba.usuario2.bookmarksutn@gmail.com, visitante = prueba.usuario1.bookmarksutn@gmail.com, password inicial = bookmarksutn.2026.

```
Credenciales rapidas para pruebas
ownerEmail=prueba.usuario2.bookmarksutn@gmail.com
ownerPassword=bookmarksutn.2026
visitorEmail=prueba.usuario1.bookmarksutn@gmail.com
visitorPassword=bookmarksutn.2026
```

1. Setear variables: ownerEmail/ownerPassword y visitorEmail/visitorPassword con esas cuentas.
2. Login owner -> crea espacio privado (se guarda espaciold).
3. Login visitor -> solicita ingreso al espacio.
4. Visitor intenta crear link con POST /api/links (idEspacio) y debe fallar con 403.
5. Login owner -> aprobar solicitud con visitorUserId.
6. Login visitor -> crear link en POST /api/links (201).
7. Visitor intenta actualizar link en PATCH /api/links/{id} (403).
8. Login owner -> actualizar link en PATCH /api/links/{id} (200).

Importante: accessToken es global en la coleccion. Cada login pisa el token anterior, por eso se alterna login owner/login visitor durante la prueba.

8) Pruebas paso a paso desde web

Opcion 1: pantalla auxiliar de pruebas

Abrir /auth-demo.html y seguir este orden: Register, Verify ultimo token, Login, /auth/me, Forgot, Reset.

La tarjeta de endpoint protegido permite probar rapidamente /api/espacios o /api/links con Bearer token.

Opcion 2: app principal

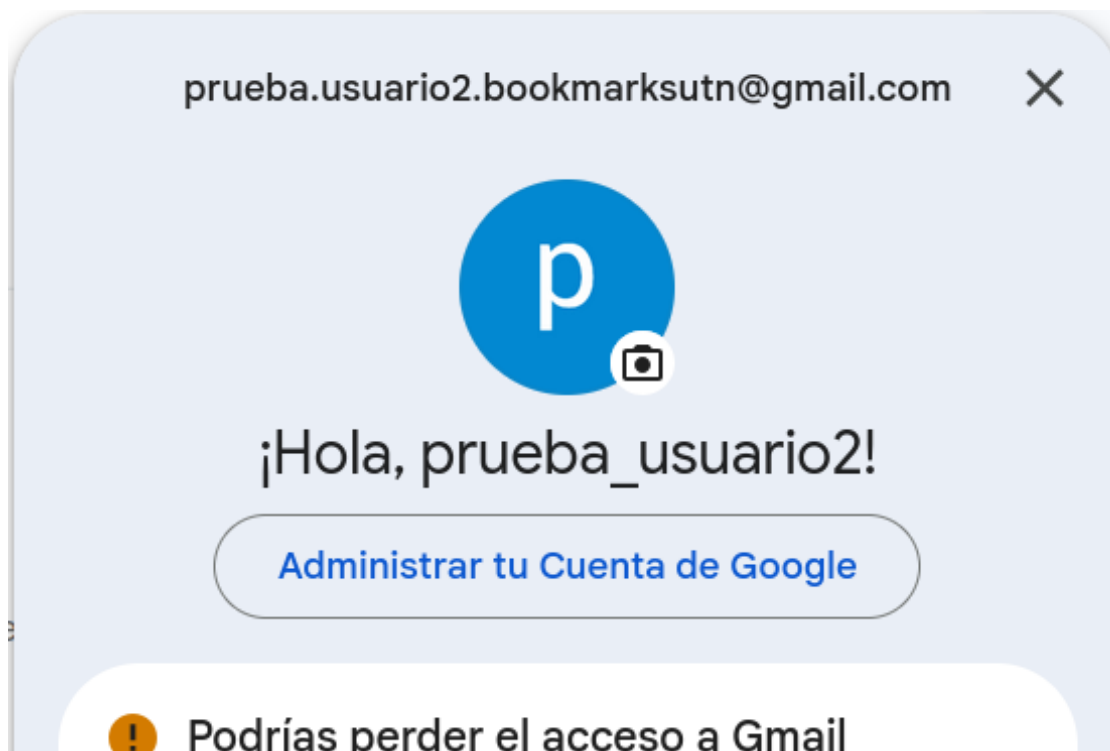
Luego de validar auth, probar comportamiento funcional por rol en la app principal: owner administra y visitante aprobado opera links segun permisos.

1. En la parte superior derecha del panel de login, usar "Sos nuevo, registrate aqui" para alta de usuario.
2. Completar nombre, email y password, confirmar desde el correo y luego iniciar sesion.
3. En login, los botones Ingresar/Cancelar quedan alineados a la derecha y debajo se muestra "Recuperar contrasena".
4. El frontend guarda accessToken y valida sesion con /api/auth/me.
5. Con sesion activa se habilitan endpoints y acciones protegidas.
6. Cerrar sesion elimina token local y la UI oculta datos hasta nuevo login.
 - Luego del registro, la UI muestra mensaje simple sin exponer token ni verifyUrlDev.
 - Al abrir verify-email desde navegador se muestra pagina HTML amigable (exito/error), no JSON plano.

9) Configuracion Gmail para demo docente

1. Crear dos cuentas Gmail dedicadas para TP (usuario1 y usuario2).
2. Activar 2FA en cada cuenta.
3. Generar App Password en la cuenta emisora desde <https://myaccount.google.com/apppasswords>.
4. Elegir Correo + dispositivo Otro (BackendUTN) y copiar la clave de 16 caracteres.
5. Configurar SMTP_SERVICE=gmail, SMTP_USER, SMTP_PASS (sin espacios), SMTP_FROM con formato nombre + email.
6. Reiniciar backend y ejecutar pruebas de email real.

Evidencia visual: correo recibido en Gmail con boton de verificacion.



10) Troubleshooting rapido

- 401 token invalido/expirado: revisar Authorization Bearer y relloguear.
- Si se uso Cerrar sesion, es normal que no carguen espacios: el token fue eliminado y se requiere nuevo login.
- 401 en login: usar No Auth, validar body email/password y revisar scope de variables activo.

- Si 401 persiste con valores hardcodedos, verificar en DB que el usuario exista, tenga passwordHash y emailVerificado=1.
- 403 en links: revisar estado en Espacio_Usuarios y ownership del espacio.
- No llega email: revisar SMTP y App Password de Gmail.
- No aparece App Password: abrir <https://myaccount.google.com/apppasswords> y reingresar luego de activar 2FA.
- Token de verify/reset invalido: puede estar vencido o ya utilizado.

11) Variables de entorno recomendadas

```
PORT=5000
MYSQL_HOST=127.0.0.1
MYSQL_PORT=3306
MYSQL_USER=root
MYSQL_PASSWORD=
MYSQL_DATABASE=bookmarks
JWT_ACCESS_SECRET=CAMBIAR_ESTE_SECRETO
JWT_ACCESS_EXPIRES=15m
BCRYPT_SALT_ROUNDS=10
EMAIL_VERIFY_TTL_HOURS=24
RESET_PASSWORD_TTL_MINUTES=30
APP_BASE_URL=http://localhost:5000
SMTP_SERVICE=gmail
SMTP_USER=prueba.usuario2.bookmarksutn@gmail.com
SMTP_PASS=app_password_de_16_caracteres_sin_espacios
SMTP_FROM="BookmarksUTN <prueba.usuario2.bookmarksutn@gmail.com>"
```

12) Preguntas de oral con respuestas modelo

Pregunta: por que JWT y no sesion en servidor?

Respuesta modelo: JWT permite backend stateless, simple de escalar y alineado con API REST. El token viaja en Authorization Bearer y el middleware valida firma y expiracion.

Pregunta: como proteges contraseñas?

Respuesta modelo: no se guardan en claro, solo hash bcrypt. En login se usa bcrypt.compare para validar.

Pregunta: como implementaste olvido de contraseña?

Respuesta modelo: forgot genera token temporal, guarda hash+expiracion, envia enlace por email y reset valida token antes de actualizar passwordHash e invalidar el token.

Pregunta: por que no envias la nueva contraseña por email?

Respuesta modelo: porque es mas seguro enviar enlace temporal con token. Asi el usuario elige su nueva clave y no circula una contraseña definitiva por correo.

Pregunta: como evitas filtrado de cuentas existentes?

Respuesta modelo: forgot-password siempre devuelve mensaje generico, exista o no el email, evitando user enumeration.

Pregunta: que pasa si roban un token viejo?

Respuesta modelo: no sirve si ya expiro o si ya fue utilizado, porque se limpia de DB al reset exitoso.

Pregunta: diferencia entre autenticacion y autorizacion?

Respuesta modelo: autenticacion define quien sos (token valido). Autorizacion define que podes hacer (owner/ aprobado/pendiente segun espacio).

Pregunta: como demostras permisos en vivo?

Respuesta modelo: visitante pendiente intenta crear link y recibe 403; owner aprueba solicitud; visitante crea link; visitante no puede modificar; owner si puede.

Mini bloque: preguntas trampa (respuesta corta)

- Si cambian idUsuario en body, pueden operar como otro? No, la identidad sale de request.user (JWT).
- Si roban un JWT, se rompe toda la seguridad? No: hay expiracion corta; mejora: refresh con revocacion.
- Por que bcrypt y no sha256 para password? Porque bcrypt tiene sal y costo adaptable.
- Si filtran DB, pueden resetear igual? No directo: se guarda hash del token, no token plano.
- El frontend define seguridad? No: la autorizacion real se decide en backend.

13) Cierre y siguientes mejoras

Estado actual: autenticacion, autorizacion por roles, JWT, verificacion de email y olvido de contrasena funcionando.

- Mejora sugerida: refresh token con revocacion.
- Mejora sugerida: tests de integracion automatizados.
- La migracion de links a idEspacio como FK real ya esta implementada.

Fin de la guia.